

1) Método `nomes(List<Planta> lista)`

O enunciado pede: escolher **aleatoriamente** uma estação (`Estacao`), construir uma `String` com os **nomes** das plantas que florescem nessa estação (separados por espaço), e devolver um `Pair<Enum, String>` com (estação, string).

Passos

1. Obter as estações: `Estacao.values()`
2. Escolher um índice aleatório `i` entre `0` e `values.length-1`
3. `Estação escolhida = values[i]`
4. Percorrer `lista` e, para cada `Planta p`:
 - se `p.estacao() == escolhida`, adicionar `p.nome()` ao `StringBuilder` (com um espaço)
5. `return new Pair<>(escolhida, sb.toString())`

```
static Pair<Enum, String> nomes(List<Planta> lista) {
    Random rnd = new Random();
    Estacao[] estacoes = Estacao.values();
    Estacao escolhida = estacoes[rnd.nextInt(estacoes.length)];

    StringBuilder sb = new StringBuilder();
    for (Planta p : lista) {
        if (p.estacao() == escolhida) {
            sb.append(p.nome()).append(" ");
        }
    }
    return new Pair<Enum, String>(escolhida, sb.toString());
}
```

2) (a) - AdegaPlus

O que acontece?

- Começa: [0, 0, 0]
- Acrescenta 20 à cuba 1 → [20, 0, 0]
- Acrescenta 25 à cuba 3 → [20, 0, 25]
- Foram feitas **2 recargas**

O que imprime?: Quantidades nas cubas: 20 0 25 Número de recargas: 2

(b) - EcoAdega

Estado inicial: [20, 30]

`a3.acrescenta(1, 3)`

- $20 + 3 = 23$ (não passa o máximo que é 30)
- Só acrescenta à cuba 1

Estado: [23, 30]

`a3.acrescenta(1, 12)`

- $23 + 12 = 35$ (passa o máximo 30)
- Então distribui 12 por todas as cubas → $12 / 2 = 6$

Novo estado: [29, 36]

2(c) - Ciclo com 3 adegas

- `Adega[] adegas = new Adega[3];`
- `adegas[0] = new Adega(2);`
- `adegas[1] = new AdegaPlus(2);`
- `adegas[2] = new EcoAdega(2, new int[] {20, 30});`

Cada uma faz:

- `acrescenta(1,8);`
- `acrescenta(2,8);`

Resultados finais

❶ Adega

- [8, 8]

Imprime: Quantidades nas cubas: 8 8

2) AdegaPlus

- [8, 8] (2 recargas)

Imprime: Quantidades nas cubas: 8 8 Numero de recargas: 2

3) EcoAdega

- Começa [20, 30]
- +8 na cuba 1 → [28, 30]
- +8 na cuba 2 → passa o máximo → distribui 8 → +4 a cada

Final: [32, 34]

Imprime:Quantidades nas cubas: 32 34

GRUPO II — (1, 2, 3) Classes ElemGraf, Poligono, Grupo

O enunciado define: **ElemGraf** abstrata com **cse()**, **cid()**, **mover(dx,dy)** abstratos e um método template **areaCaixa()** que calcula a área a partir de **cse** e **cid**. Também pede **mudaCor**.

1) ElemGraf (abstrata)

areaCaixa():

- largura = cid().x() - cse().x()
- altura = cse().y() - cid().y()
- área = largura * altura

```
public abstract class ElemGraf {
    private Color cor;

    public ElemGraf(Color c) {
        this.cor = c;
    }

    public void mudaCor(Color c) {
        this.cor = c;
    }

    public Color cor() {
        return this.cor;
    }

    public int areaCaixa() {
        Ponto cse = cse();
        Ponto cid = cid();
        int largura = cid.x() - cse.x();
        int altura = cse.y() - cid.y();
        return largura * altura;
    }

    public abstract Ponto cse();
    public abstract Ponto cid();
    public abstract void mover(int dx, int dy);

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || this.getClass() !=
            o.getClass()) return false;
        ElemGraf other = (ElemGraf) o;
        return this.cor.equals(other.cor)
            && this.cse().equals(other.cse())
            && this.cid().equals(other.cid());
    }
}
```

2) Poligono

Pede: lista de vértices, cse = (menor x, maior y), cid = (maior x, menor y), e mover move todos os pontos.

```
public class Poligono extends ElemGraf {
    private List<Ponto> vertices;

    public Poligono(Color c, List<Ponto> l) {
        super(c);
        this.vertices = new
            ArrayList<Ponto>(l);
    }

    @Override
    public Ponto cse() {
        int minX = vertices.get(0).x();
        int maxY = vertices.get(0).y();
        for (Ponto p : vertices) {
            if (p.x() < minX) minX = p.x();
            if (p.y() > maxY) maxY = p.y();
        }
        return new Ponto(minX, maxY);
    }

    @Override
    public Ponto cid() {
        int maxX = vertices.get(0).x();
        int minY = vertices.get(0).y();
        for (Ponto p : vertices) {
            if (p.x() > maxX) maxX = p.x();
            if (p.y() < minY) minY = p.y();
        }
        return new Ponto(maxX, minY);
    }

    @Override
    public void mover(int dx, int dy) {
        for (Ponto p : vertices) {
            p.mover(dx, dy);
        }
    }

    @Override
    public boolean equals(Object o) {
        if (!super.equals(o)) return false;
        Poligono other = (Poligono) o;
        return
            this.vertices.equals(other.vertices);
    }
}
```

3) Grupo

Pede: cor do grupo é `UtilsDesenho.calculaCor(l)`, `cse/cid` por agregação, mover move tudo, e redefinir `mudaCor` para também mudar a cor dos elementos.

```
public class Grupo extends ElemGraf {
    private List<ElemGraf> elems;

    public Grupo(List<ElemGraf> l) {
        super(UtilsDesenho.calculaCor(l));
        this.elems = new ArrayList<ElemGraf>(l);
    }

    @Override
    public Ponto cse() {
        Ponto first = elems.get(0).cse();
        int minX = first.x();
        int maxY = first.y();
        for (ElemGraf e : elems) {
            Ponto p = e.cse();
            if (p.x() < minX) minX = p.x();
            if (p.y() > maxY) maxY = p.y();
        }
        return new Ponto(minX, maxY);
    }

    @Override
    public Ponto cid() {
        Ponto first = elems.get(0).cid();
        int maxX = first.x();
        int minY = first.y();
        for (ElemGraf e : elems) {
            Ponto p = e.cid();
            if (p.x() > maxX) maxX = p.x();
            if (p.y() < minY) minY = p.y();
        }
        return new Ponto(maxX, minY);
    }

    @Override
    public void mover(int dx, int dy) {
        for (ElemGraf e : elems) {
            e.mover(dx, dy);
        }
    }

    @Override
    public void mudaCor(Color c) {
        super.mudaCor(c);
        for (ElemGraf e : elems) {
            e.mudaCor(c);
        }
    }

    public List<ElemGraf> elementos() {
        return new ArrayList<ElemGraf>(elems);
    }
}
```

5 - b) Implementar **imprimeAreasCaixas**

```
static void imprimeAreasCaixas(Collection<? extends ElemGraf> col) {
    for (ElemGraf e : col) {
        System.out.println(e.areaCaixa());
    }
}
```